4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Course: Digital VLSI Design
Technology: SkyWater SKY130 (130 nm), standard cell library sky130_fd_sc_hd

Name	Roll number
Vedant Pahariya	2023112012
N Sujith Kumar	2025802004

All source files, scripts and generated reports are in the repository; the commands used for every step are listed in README.md. GitHub link: https://github.com/sujithiiith-byte/dvd-array-multiplier

---

Table of Contents

Part A: RTL Design and Verification 2
A.1 Block diagram 2
A.2 Simulation results 2
A.3 Hand-worked example (task 5) 3
A.4 Waveform and latency 4
Part B: Logic Synthesis with Yosys 4
B.1 Results 4
B.2 Gate-level (netlist) simulation 4
Part C: Static Timing Analysis with OpenSTA 4
C.1 Finding the maximum frequency 4
C.2 Results 5
C.3 Cells on the critical path (T = 2.60 ns) 5
C.4 Comparison with the 2N estimate 6
Part D: Physical Design (RTL to GDSII) with LibreLane 6
D.1 Clock period and attempts 6
D.2 Results (run 8, pnr/metrics.json) 7
D.3 Cells added by the flow 7
D.4 Layout 8
Answers to the Questions (Section 10) 9

Part A: RTL Design and Verification

A.1 Block diagram

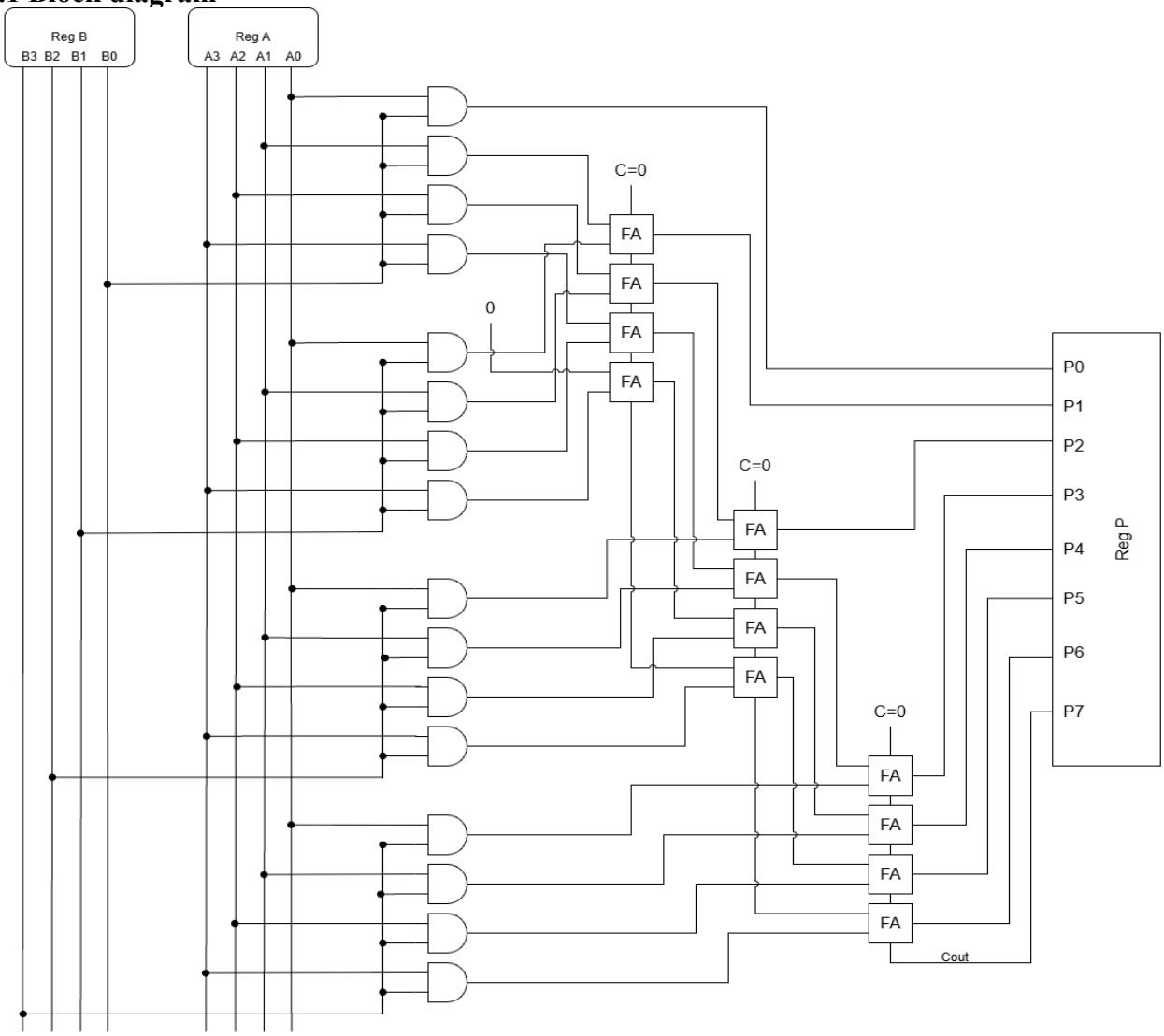


Figure A.1: Block diagram. Reg A / Reg B (input registers), 16 AND gates (4 per partial product), 12 full adders in three ripple-carry rows (carry-in C = 0), and Reg P (output register).

A.2 Simulation results

RTL simulation:

PASS: 256 vectors checked, 0 errors

```
/foss/designs > iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
/foss/designs > vvp sim
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:68: $finish called at 5140000 (1ps)
```

Figure A.2: RTL simulation in the container: PASS: 256 vectors checked, 0 errors.

Error-detection test (task 6). In rtl/full_adder.v the sum was temporarily changed to assign sum = a ^ b; (carry-in ignored). The testbench reported (first 3 of the 95 mismatch lines, then the summary line):

```
/foss/designs > vvp sim
VCD info: dumpfile mult.vcd opened for output.
MISMATCH: a=3 b=3 p=5 expected=9
MISMATCH: a=3 b=6 p=10 expected=18
MISMATCH: a=3 b=7 p=9 expected=21
```

Figure A.3: Error test, start of the output (first mismatches).

```
MISMATCH: a=15 b=14 p=154 expected=210
MISMATCH: a=15 b=15 p=181 expected=225
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:68: $finish called at 5140000 (1ps)
```

Figure A.4: Error test, end of the output: last mismatches and the summary line FAIL: 256 vectors checked, 95 errors.

A.3 Hand-worked example (task 5)

Roll number used:

Vedant Pahariya, 2023112012 → last digit 2 → **A = 2 + 5 = 7**. Since A = 7, **B = 9**. A × B = 7 × 9 = 63 = 00111111.

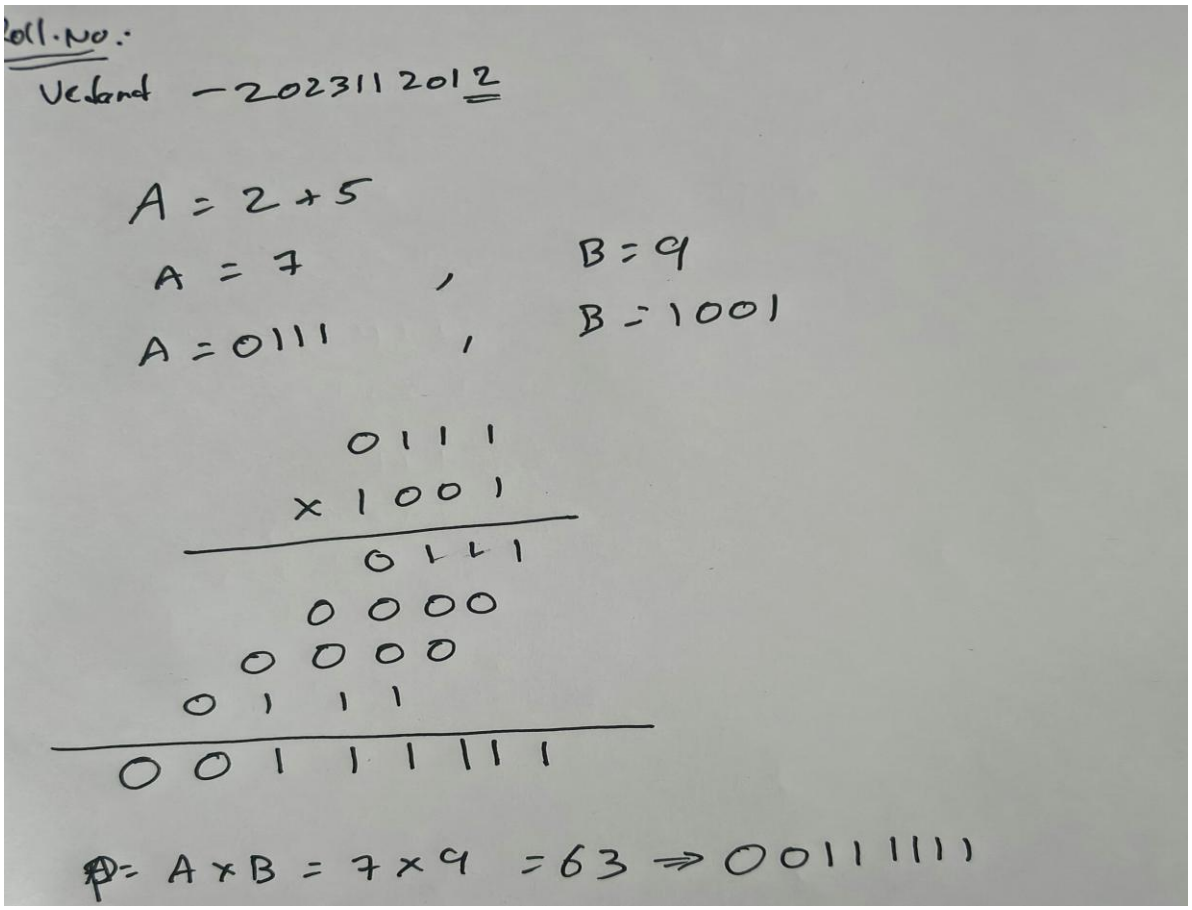


Figure A.5: Hand-worked example, 7 × 9 = 63 = 0011 1111₂.

A.4 Waveform and latency

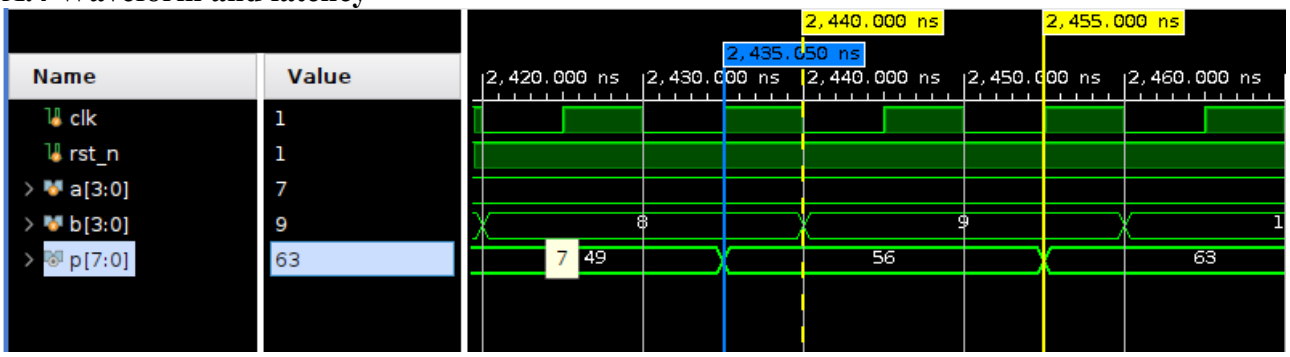


Figure A.6: Waveform for A = 7, B = 9. Cycle 1 starts at the rising edge at 2435 ns (blue marker). The testbench applies a = 7, b = 9 on the falling edge at 2440 ns (dashed marker); rising edge 1 at 2445 ns captures them into a_reg and b_reg; at rising edge 2, 2455 ns (marker), the product p = 63 is loaded. The latency is therefore 2 clock cycles (2435 → 2455 ns).

Part B: Logic Synthesis with Yosys

B.1 Results

Metric	Value
Total number of cells	73
Number of flip-flops	16 (sky130_fd_sc_hd__dfrtp_1, 25.024 µm² each)
Total cell area	805.773 µm²
Flip-flop area	16 × 25.024 = 400.384 µm²
Flip-flop area (% of total)	49.69 %
The 3 most-used cell types	dfrtp_1 × 16, nand2_1 × 11, then a four-way tie at × 4: a21oi_1, and2_0, nand4_1, xnor2_1
Netlist simulation result	PASS: 256 vectors checked, 0 errors

B.2 Gate-level (netlist) simulation

The same exhaustive testbench was run against the synthesized netlist, using the SKY130 cell models, to show that synthesis did not change the behaviour.

```
/foss/designs > iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
  tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
/foss/designs > vvp gls
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:68: $finish called at 5140000 (1ps)
```

Figure B.1: Gate-level simulation of the synthesized netlist: PASS: 256 vectors checked, 0 errors.

Part C: Static Timing Analysis with OpenSTA

C.1 Finding the maximum frequency

For a flip-flop-to-flip-flop path, slack = (T − setup time) − arrival time. The arrival time does not depend on T, so the slack shrinks one-for-one with the period:

Step	Result
Run at T = 10 ns	worst slack **7.40 ns** (7.404 ns in the report)
T_min = T − WNS = 10 − 7.40	**2.60 ns**
F_max = 1000 / T_min	**384.6 MHz**
Run again at T = 2.60 ns	worst slack **0.00 ns** (~0.004 ns in the report), confirming T_min

C.2 Results

Metric	Value
Worst slack at T = 10 ns	7.40 ns
T_min	2.60 ns
F_max	384.6 MHz
Slack at T = T_min	0.00 ns (~0.004 ns)
Critical path start point	b_reg[0] (flip-flop _118_, sky130_fd_sc_hd__dfrtp_1, rising edge of clk)
Critical path end point	p[6] (flip-flop _112_, D pin; net fa_3_3.sum)
Number of cells on the critical path	10: 2 flip-flops (launch and capture) and 8 logic cells
Latency = 2 × T_min	5.20 ns
Throughput	384.6 million results per second (one result per clock cycle)

The flip-flop names _118_ and _112_ are generated by Yosys; the registers were identified from the .Q(...) connections in synth/mult_array_netlist.v.

C.3 Cells on the critical path (T = 2.60 ns)

#	Instance	Cell	Delay (ns)	Arrival (ns)	Function
1	_118_	dfrtp_1 (CLK → Q)	0.395	0.395	launches b_reg[0]
2	_083_	and2_0	0.212	0.607	partial product pp0[0] = a_reg[0] & b_reg[0]
3	_055_	or3_1	0.361	0.968	carry logic
4	_059_	and4_1	0.215	1.183	carry logic
5	_070_	o2111a_1	0.234	1.417	carry logic
6	_076_	o21ai_0	0.216	1.634	carry logic
7	_090_	xnor3_1	0.274	1.907	full-adder sum function
8	_091_	maj3_1	0.375	2.282	full-adder carry function
9	_092_	xor3_1	0.178	2.460	full-adder sum → p[6]
10	_112_	dfrtp_1 (setup)	0.135	required 2.465	captures p[6]

Data arrival 2.460 ns + setup 0.135 ns = 2.595 ns ≈ T_min. The report also lists a path in the “asynchronous” group (rst_n → RESET_B): this is a reset recovery check with 1.828 ns of slack, not the critical data path.

C.4 Comparison with the 2N estimate

Section 2 estimates that the longest path through an N-bit array crosses about 2N = 8 full adders. In our RTL array this is exact: the slowest route goes through 2 adders in row 1, 2 in row 2 and all 4 in row 3 (the carry ripple along the last row), 8 full adders in total, and ends at p[6]/p[7].

After synthesis the same path is made of 8 logic cells instead of 8 separate full adders: 1 AND gate (the partial product) and 7 cells doing full-adder work. Three of them are full-adder functions (xnor3 and xor3 for the sum, maj3 for the carry); the other four (or3, and4, o2111a, o21ai) are complex gates into which ABC merged the carry logic of several adders, because -flatten removes the adder boundaries.

Part D: Physical Design (RTL to GDSII) with LibreLane

D.1 Clock period and attempts

The starting period was 1.2 × T_min = 1.2 × 2.60 = 3.12 ns, rounded up to the next 0.5 ns: **3.5 ns**. Every run finished with 0 DRC and 0 LVS errors but stopped with setup violations, so the period was increased by 0.5 ns per attempt:

Run	CLOCK_PERIOD (ns)	Setup slack, all corners (ns)	Setup slack, tt corner (ns)	DRC	LVS	Result
1	3.5	−2.154	+0.587	0	0	failed (setup, ss corner)
2	4.0	−1.654	+1.087	0	0	failed (setup, ss corner)
3	4.5	−1.154	+1.587	0	0	failed (setup, ss corner)
4	5.0	−0.654	+2.087	0	0	failed (setup, ss corner)
5	5.5	−0.155	+2.587	0	0	failed (setup, ss corner)
6	6.0	−0.107	+2.899	0	0	failed (setup, ss corner)
7	6.5	−0.136	+3.199	0	0	failed (setup, ss corner)
8	**7.0**	**+0.364**	+3.699	0	0	**passed**

Part C used only the typical corner (tt, 25 °C, 1.80 V), where even the first run had +0.59 ns of slack. LibreLane signs off at nine corners (tt, ss and ff process/voltage/temperature corners, each with minimum, nominal and maximum wire parasitics). The violations were always at the **slow corner (ss, 100 °C, 1.60 V)**, where the cells are about 2.5 times slower. From 3.5 ns to 5.5 ns the layout was identical and the slack improved by exactly 0.5 ns per step. At 6.0 ns and 6.5 ns it stalled: with more slack available, the resizer downsized logic cells to save area (logic cell area 395 → 368 → 327 µm²), which slowed the slow-corner path again. At 7.0 ns the design met timing at all corners.

D.2 Results (run 8, pnr/metrics.json)

Metric	Value
CLOCK_PERIOD used	7.0 ns
Number of runs needed	8 (3.5 ns → 7.0 ns in 0.5 ns steps, Section D.1)
Core area	17 759.5 µm² (die area 150 × 150 = 22 500 µm²)
Cells after synthesis (from Part B)	73
Cells after place and route	1895 (+1822)
Kinds of cells added	decap (sky130_fd_sc_hd__decap_3/4/6/8/12) 1314, fill (fill_1, fill_2) 247, tap (tapvpwrvgnd_1) 245, buffers: I/O buf_2 17, clock tree clkbuf_16 3, reset fan-out clkdlybuf4s50_1 2
Wirelength	2362 µm
Setup slack	+0.364 ns (worst of all nine corners, at max_ss_100C_1v60); +3.699 ns at the typical corner
Post-route F_max	1000 / (7.0 − 0.364) = 150.7 MHz
F_max from Part C	384.6 MHz
DRC errors	0 (Magic 0, KLayout 0)
LVS errors	0

D.3 Cells added by the flow

Kind	Count	Cells	Purpose
decap	1314	decap_3/4/6/8/12	decoupling capacitance between VPWR and VGND in the empty row space
fill	247	fill_1, fill_2	fill the remaining gaps so the rows (wells and power rails) are continuous
tap	245	tapvpwrvgnd_1	connect n-well to VPWR and substrate to VGND at regular spacing (latch-up prevention)
I/O buffers	17	buf_2	one per input (a, b, rst_n: 9) and output (p: 8)
clock tree	3	clkbuf_16	clock tree synthesis: one root buffer driving two leaf buffers, 8 flip-flops each
fan-out buffers	2	clkdlybuf4s50_1	split the rst_n net, which drives all 16 flip-flops
flip-flops	16	dfrtp_1	from synthesis
logic	51	AND/OR/XOR/complex gates	from LibreLane's own synthesis (67 cells including the 16 flip-flops)

D.4 Layout

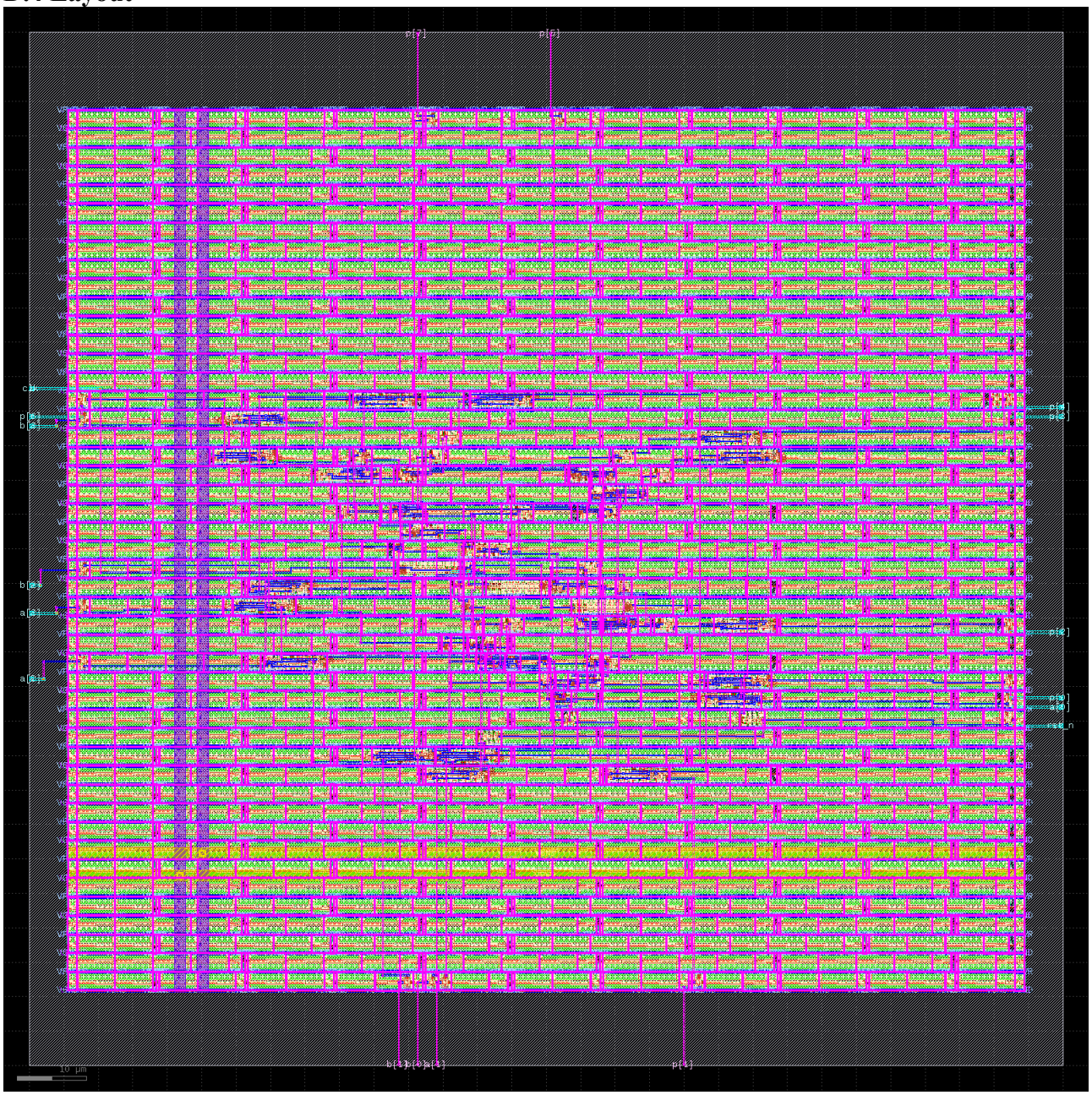


Figure D.1: Final layout of the whole 150 µm × 150 µm die in KLayout: I/O pins on the edges, standard-cell rows, the metal power grid, and the logic in the center.

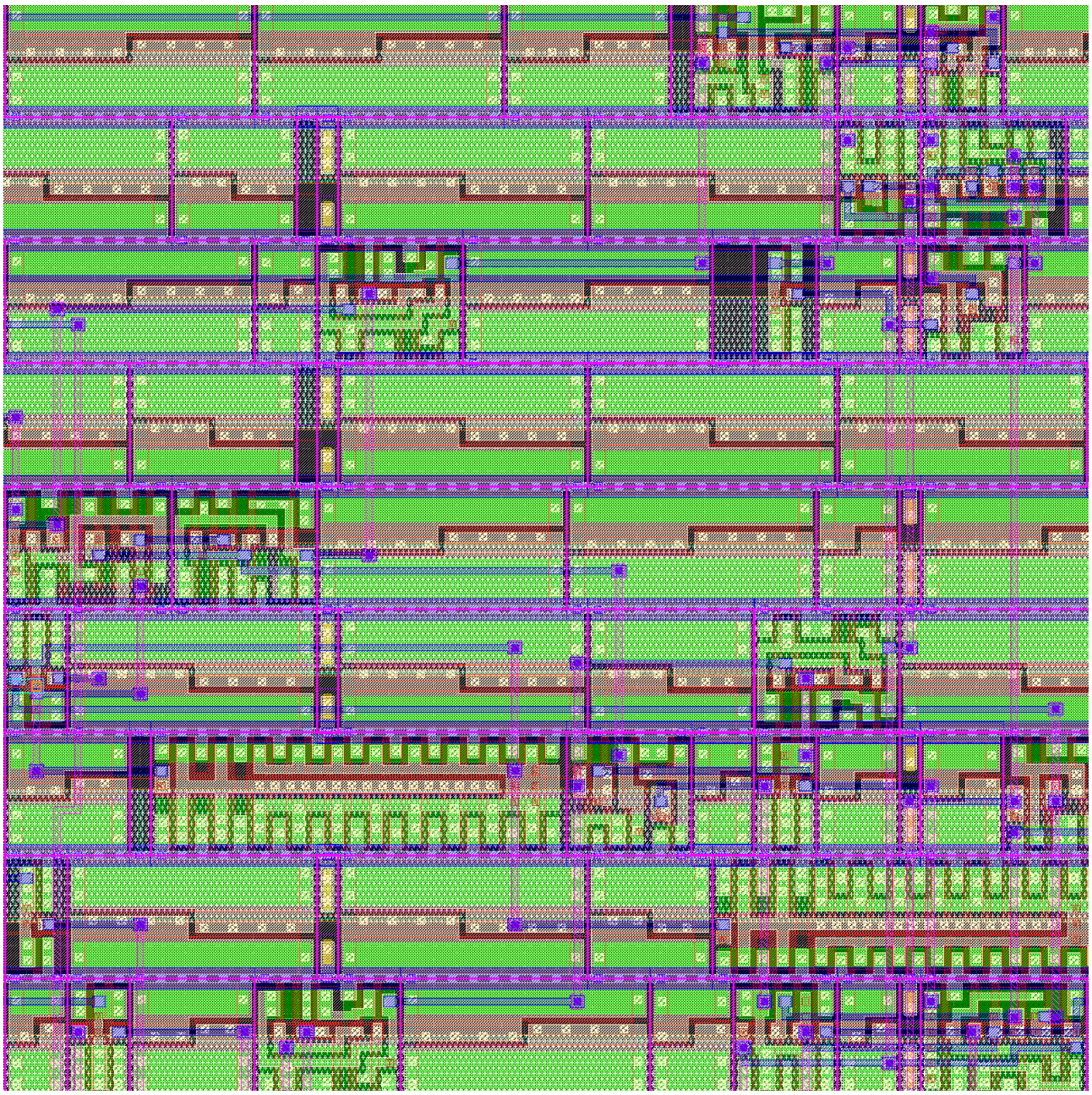


Figure D.2: Zoomed view of the center of the core: standard-cell rows with logic cells, decap/fill cells, metal routing and vias.

Answers to the Questions (Section 10)

Q1. Cells for the logic, and why not simply 16 + 12. Synthesis produced 73 cells in total, including 16 flip-flops, so 57 cells implement the combinational logic. The count is not simply 16 AND gates plus 12 full adders because a full adder does not necessarily map to a single standard cell. During synthesis, ABC optimizes the flattened logic, simplifies parts with constant inputs, and combines some functions into complex gates. As a result, the final cell count depends on the standard-cell library and the optimizations performed, rather than directly matching the number of gates in the RTL. For example, the library's full-adder cell (sky130_fd_sc_hd__fa_1) has two outputs, which ABC cannot use, so each adder's sum and carry are built from separate gates; adders with a constant-0 input (the first adder of each row, whose carry-in is 0, and fa_1_3, whose input r0[3] is 0) reduce to half adders; the carry logic reuses the XOR of the sum; and AND gates and carry logic are merged into complex cells such as and4, o2111a, a21oi and maj3.

Q2. Critical path. The critical path runs from the input register **b_reg[0]** to the output register **p[6]** (the sum output of fa_3_3). It contains **10 cells**: the launching and capturing flip-flops and 8 logic cells (and2_0, or3_1, and4_1, o2111a_1, o21ai_0, xnor3_1, maj3_1, xor3_1), with a data arrival of 2.460 ns. In the RTL array the longest path crosses 8 full adders, matching the estimate of 2N = 8. Because we flattened the design during synthesis, ABC merged parts of the carry chain into complex gates, leaving 1 partial-product AND gate and 7 adder-related logic cells along the path.

Q3. Flip-flop share of the area. The 16 flip-flops occupy 400.384 µm² of 805.773 µm², i.e. **49.69 %** of the total cell area. One dfrtp_1 is 25.02 µm², about 7 times a nand2_1 (3.75 µm²), because a flip-flop with asynchronous reset contains two latches (master and slave), clock buffering and reset logic, roughly 20–30 transistors against 4 for a NAND2. The multiplier logic is tiny (57 small cells), so 16 flip-flops weigh as much as all of it together. For larger multipliers the logic grows roughly with N^2 while the registers grow only with 4N, so the flip-flop share would drop.

Q4. Post-route F_max and added cells. The post-route maximum frequency is 150.7 MHz, lower than the 384.6 MHz obtained from static timing analysis in Part C. One reason is that Part C uses only the typical operating corner, whereas LibreLane checks nine corners and must satisfy timing at the slowest one. Physical routing also introduces wire resistance and capacitance, while clock-tree buffers, clock uncertainty and other added cells affect timing. The flow added 1822 cells (73 → 1895): 1314 decap cells, which store charge locally so the supply stays steady when gates switch; 247 fill cells, which keep the rows, wells and power rails continuous; 245 tap cells, which tie the n-well and substrate to the supplies to prevent latch-up; and 22 buffers: 17 for the I/O pins, 3 for the clock tree to keep skew low, and 2 to split the reset net that drives all 16 flip-flops. At the typical corner, the estimated post-route maximum frequency would be 302.9 MHz, showing that routing and implementation overhead also contribute to the reduction.

Q5. DRC and LVS. DRC (design rule check) verifies the layout geometry against the foundry's manufacturing rules: minimum width and spacing of every layer, via enclosures, density and similar rules. A violation means the shapes may not be fabricated correctly (shorts, opens). **LVS (layout versus schematic)** extracts the transistors and connections from the layout and compares them with the netlist, proving that the layout implements exactly the intended circuit, with no missing, extra or wrongly connected devices. Both checks are mandatory before tape-out because a DRC-clean layout can still have a miswired net or missing gate, and an LVS-clean circuit can still violate geometric spacing rules and fail in silicon. Our final GDSII passed with 0 DRC errors in Magic and KLayout and 0 LVS errors.

Q6. Scaling to 8 bits.

Full adders: $N(N-1) \rightarrow 4 \times 3 =$ **12** for N = 4 and $8 \times 7 =$ **56** for N = 8

This is about 4.7 times more full adders (AND gates: $N^2 = 16 \rightarrow 64$). Critical path: about 2N full-adder delays → **8** for N = 4 and **16** for N = 8, i.e. twice as long, because the delay grows linearly with N.

Using our results: the logic part of the critical path is 2.460 − 0.395 = 2.065 ns and the flip-flop overhead (clock-to-Q 0.395 ns + setup 0.135 ns) is 0.53 ns, so for 8 bits $T_{min} \approx 0.53 + 2 \times 2.065 \approx$ **4.7 ns** ($F_{max} \approx 215$ MHz) instead of 2.60 ns (384.6 MHz). This is a rough estimate: synthesis restructuring and larger fan-out change the exact value.

Individual Contributions

N Sujith Kumar (2025802004): Set up the tool environment (IIC-OSIC-TOOLS Docker image with the SKY130 PDK) and the Git repository. Part A: wrote the full adder and array multiplier RTL and the exhaustive self-checking testbench, ran the RTL simulation and the error-detection test, and captured the waveform showing the 2-cycle latency. Part B: wrote the Yosys synthesis script, analysed the cell and area report, and verified the synthesized netlist with gate-level simulation.

Vedant Pahariya (2023112012): Part C: wrote the OpenSTA script, determined T_min and F_max, identified the critical path and compared it with the 2N estimate. Part D: configured and ran the LibreLane RTL-to-GDSII flow, increased the clock period over eight runs until timing was met at all corners, verified DRC and LVS, analysed the cells added by the flow and the post-route F_max, and captured the layout screenshots.

Both: Reviewed each other's parts and the final report.